Re-secuenciamiento por color en plantas de pintura: reproducción de HAAM-RL y búsqueda guiada por redes de valor

Equipo HAAM-RL Twin · 13/10/2026 · Informe técnico reproducible

Resumen

Reproducimos HAAM-RL (Choi et al., 2024), un agente de aprendizaje por refuerzo con máscara heurística para reordenar vehículos antes de la cabina de pintura y minimizar los cambios de color, y lo comparamos con familias de métodos completamente distintas. Tras corregir un gemelo digital que no simulaba la misma planta que FlexSim, la heurística del paper queda en 31,73 cambios por cada 100 vehículos (500 escenarios de test). El agente RL al estilo del paper la reduce un 2,0 % y la mejor configuración RL, un 7,6 %. Mostramos que añadir capas a la red no ayuda, que una red que respeta la simetría entre buffers sí, y que el límite real está en cómo se estima el coste futuro. La mejor política es un planificador por búsqueda en haz que, conociendo el vehículo entrante y las 10 llegadas siguientes, evalúa el futuro con el promedio de 8 redes de valor: 16,83 cambios, una reducción del 46,9 % (IC 95 % 46,4–47,5 %), mejor que la heurística en 500 de 500 escenarios y a 2,58 cambios de un oráculo que conoce las 100 llegadas. Esa cifra supone que la planta conoce la secuencia de los 10 vehículos siguientes. Si solo se conoce el próximo vehículo, como en el agente del paper, el mismo planificador sin anticipación obtiene 24,34 cambios (23,3 % menos que la heurística, mejor en 493 de 500 escenarios) y decide unas 10 veces más rápido (sección 5.10). Cada cifra de este documento se genera a partir de los resultados guardados, con semillas fijas y pruebas pareadas.

1. Introducción

Cada cambio de color en la cabina obliga a purgar las pistolas: cuesta pintura, solvente y tiempo. El problema de re-secuenciamiento por color (CRP) consiste en usar buffers intermedios para agrupar los vehículos del mismo color. Choi et al. proponen HAAM-RL: un agente Soft Actor-Critic cuyas acciones se filtran con las heurísticas LP, CM y UCM, con recompensa con potencial (PBRS) y un ensemble de modelos en inferencia. Reportan 29,57 cambios frente a 34,38 de la heurística (–14,0 %; el paper indica 16,25 % porque su ecuación 5 divide por el resultado del agente). Las tablas de su apéndice suman, sin embargo, medias de 34,57 y 29,90.

Este trabajo tiene tres objetivos: reproducir el método con mediciones exactas y reproducibles; entender qué limita su rendimiento (arquitectura, entrenamiento, máscara, información disponible); y evaluar enfoques completamente distintos, incluida la combinación de varios modelos que deciden juntos. Las contribuciones son:

- Un gemelo digital alineado con FlexSim, con 55 pruebas automáticas y un verificador de trazas que detecta cualquier divergencia entre ambos.
- Una reproducción del agente del paper y un estudio de arquitectura: más capas no mejoran; una red equivariante a permutaciones de buffers sí.
- La medición del techo de cualquier política que obedezca la máscara del paper, y la comprobación de que el cuello de botella era el aprendizaje.
- Un planificador por búsqueda en haz guiado por un ensemble de redes de valor, que es la mejor política encontrada, y su destilación en una red rápida.
- Una evaluación sistemática de «varios modelos a la vez»: votación (no ayuda), comités arbitrados por simulación (solo ayudan con un buen juez) y ensembles de jueces (la mejora más clara).

2. Planta y formulación

La planta tiene una cola de llegada, 5 buffers FIFO de 5 plazas y la cabina. Cada escenario son 100 vehículos de 6 colores con la mezcla del paper (6, 38, 29, 14, 10 y 3 %). Cada decisión es un par (buffer de entrada, buffer de salida), 25 acciones, y su efecto depende de la fase:

- **Llenado** (ocupación < 25): el vehículo entrante se guarda en el buffer i; acción (i, i).
- **Régimen** (buffers llenos): se pinta el frente del buffer o y el entrante se guarda en i; con buffers llenos solo cabe en la plaza liberada.
- **Vaciado** (sin llegadas): se pinta el frente del buffer o; acción (o, o).

La métrica es el número de cambios de color en el episodio. La observación del agente usa la del paper (27 valores) o una codificación one-hot con la fase, la ocupación y las k llegadas siguientes (anticipación). La recompensa es la del paper: −0,1 por cambio, 0,1 + 0,02 × racha por mantener el color, más PBRS con potencial nulo en el estado final (condición que garantiza no alterar la política óptima).

3. Métodos comparados

Heurística. Reglas del paper: para guardar, CM (cola del mismo color) y si no LP (buffer con menos colores distintos); para pintar, CM (frente del último color) y si no UCM (buffer con más colores distintos); los empates van al buffer superior.

Aprendizaje por refuerzo (paper). MaskablePPO con la máscara HAAM: la política solo puede elegir acciones que las reglas anteriores también propondrían. Probamos entrada cruda y one-hot, anticipación 0 y 10, máscaras HAAM, solo de salida y física, y redes MLP de 2 y 3 capas de 64 a 512 neuronas.

Red estructurada. Los 5 buffers son intercambiables: permutarlos permuta las acciones. Nuestra red procesa cada buffer con los mismos pesos, puntúa cada par (entrada, salida) con una función compartida y es exactamente equivariante, cosa que un MLP debe aprender de los datos (120 ordenaciones).

Imitación (DAgger). La red aprende a reproducir las decisiones de un planificador; en cada iteración juega ella misma y el planificador etiqueta los estados a los que llega (Ross et al., 2011). Opcionalmente, PPO refina el resultado.

Planificador por búsqueda en haz. En cada decisión simula todas las acciones físicamente posibles sobre los vehículos conocidos (el entrante y los 10 siguientes, o solo el entrante si la planta no conoce la cola; sección 5.10), conserva los 32 mejores planes por profundidad y ejecuta la primera acción del mejor. Un plan se puntúa con sus cambios de color más una estimación del coste futuro del estado que deja: 0,2 × fragmentación de los buffers (fórmula manual) o una red de valor.

Búsqueda guiada por valor. Una red de valor, invariante al orden de los buffers, estima cuántos cambios habrá en los 30 vehículos siguientes viendo solo el contenido de los buffers y el último color (lo que se sabe al final del horizonte conocido). Se entrena con partidas del planificador en escenarios disjuntos de validación y test. El planificador usa cambios + λ × valor; con varias redes, su promedio («ensemble de jueces»).

Varios modelos a la vez. (a) *Votación* del paper: cada red vota y gana la mayoría (o la suma de probabilidades). (b) *Rollout* (Bertsekas): para cada acción posible se simula el horizonte conocido siguiendo a una política base y se elige la mejor. (c) *Comité*: el planificador y rollouts con distintas políticas base proponen acciones con su puntuación simulada; un árbitro común elige. (d) *Ensemble de jueces*: varias redes de valor promediadas dentro de la búsqueda.

4. Protocolo experimental

- **Semillas disjuntas.** Entrenamiento con semillas aleatorias o ≥ 10^6; validación (selección de modelos e hiperparámetros) con 100 escenarios (20000–20099); test con los 30 escenarios del tamaño del protocolo del paper (10000–10029) y con 500 escenarios ampliados (30000–30499), que son los que se reportan como resultado principal (error estándar ≈ 0,15 cambios).
- **Estadística pareada.** Todas las políticas se evalúan en los mismos escenarios. Se reporta el intervalo bootstrap del 95 % (remuestreo de escenarios) de la media y de la reducción relativa, y el valor p de un test de permutación de signos a dos colas (100.000 remuestreos). Para RL, cada fila promedia por escenario las semillas entrenadas, es decir, el resultado esperado de un entrenamiento.
- **Exactitud.** El entrenamiento es determinista (misma semilla, pesos idénticos). Cada optimización del código se verificó reproduciendo bit a bit todos los resultados guardados. Las estimaciones de valor se redondean a 10^{-5} para que las decisiones no dependan del reparto entre procesos.

- **Cómputo.** CPU, un núcleo por entrenamiento; ~3.000 pasos por segundo con MLP 64×64. Todo el trabajo cabe en una estación de 16 núcleos.

5. Iteraciones y resultados

5.1 Corregir la planta antes de medir

La primera medición daba una heurística de 44,4 cambios, muy lejos de los 34,38 del paper: el gemelo nunca llenaba los buffers. Con la planta corregida la heurística queda en 32,33 en los 30 escenarios del tamaño del paper y en 31,73 en los 500 ampliados, coherente con el paper.

Tabla 1. Errores corregidos en la implementación inicial y en el código de FlexSim.

Problema encontrado	Efecto	Corrección
El gemelo nunca tenía más de 5 vehículos en 25 plazas	heurística en 44,4 cambios en vez de ~34	fases de llenado, régimen y vaciado
FlexSim contaba cambios de vehículos que no pintaba	KPI inflado y duplicado	el KPI se actualiza tras mover a la cabina
FlexSim decidía cuándo extraer con reglas propias	la API y FlexSim simulaban cosas distintas	la API dicta fase, extracción e inserción
Reglas de entrada evaluadas antes de extraer	candidatos incorrectos en régimen	se evalúan tras la extracción
Potencial PBRS no nulo al final	el shaping podía cambiar la política óptima	Φ(final) = 0, verificado por test
Selección del mejor modelo por recompensa media	el «mejor» no era el de menos cambios	selección por cambios en validación
Caché de valores que podía perder entradas	fallos esporádicos en búsquedas largas	caché LRU corregida y probada

5.2 Reproducción del agente del paper

Con 1M de pasos por semilla, el agente con máscara HAAM mejora la heurística de forma significativa pero modesta. La codificación one-hot ayuda; la anticipación casi no ayuda al agente; y sin la máscara heurística PPO no aprende en el mismo presupuesto, lo que confirma la tesis central del paper. El ensemble con la votación del paper da 30,28 frente a 29,62 de media de sus miembros: no mejora.

Tabla 2. Reproducción del agente RL, 500 escenarios de test (última columna: los 30 del tamaño del paper).

Configuración (PPO)	Media	IC 95 %	Reducción	IC reducción	p	G/E/P	Media (30)
Entrada cruda, como el paper	31,11	30,88–31,35	2,0 %	1,2 a 2,7 %	< 0,0001	291/5/204	31,15
One-hot	29,62	29,36–29,88	6,7 %	5,9 a 7,4 %	< 0,0001	369/13/118	30,57
One-hot + anticipación 10	30,09	29,84–30,33	5,2 %	4,4 a 5,9 %	< 0,0001	347/9/144	30,49
Sin máscara heurística	41,26	40,95–41,57	-30,0 %	-31,1 a -28,9 %	< 0,0001	0/1/499	41,19

5.3 ¿Más capas? No: más estructura

Entrenamos MLP de 2 y 3 capas de 64 a 256 neuronas durante 3M de pasos. Ninguna supera a la 64×64, y la más grande es la peor. Para separar capacidad de dificultades de RL medimos cuánto puede imitar cada red las decisiones del planificador en escenarios no vistos: los MLP se quedan en torno al 59 % de acierto en las decisiones reales aunque memoricen el entrenamiento, mientras la red estructurada llega al 67 % con menos parámetros.

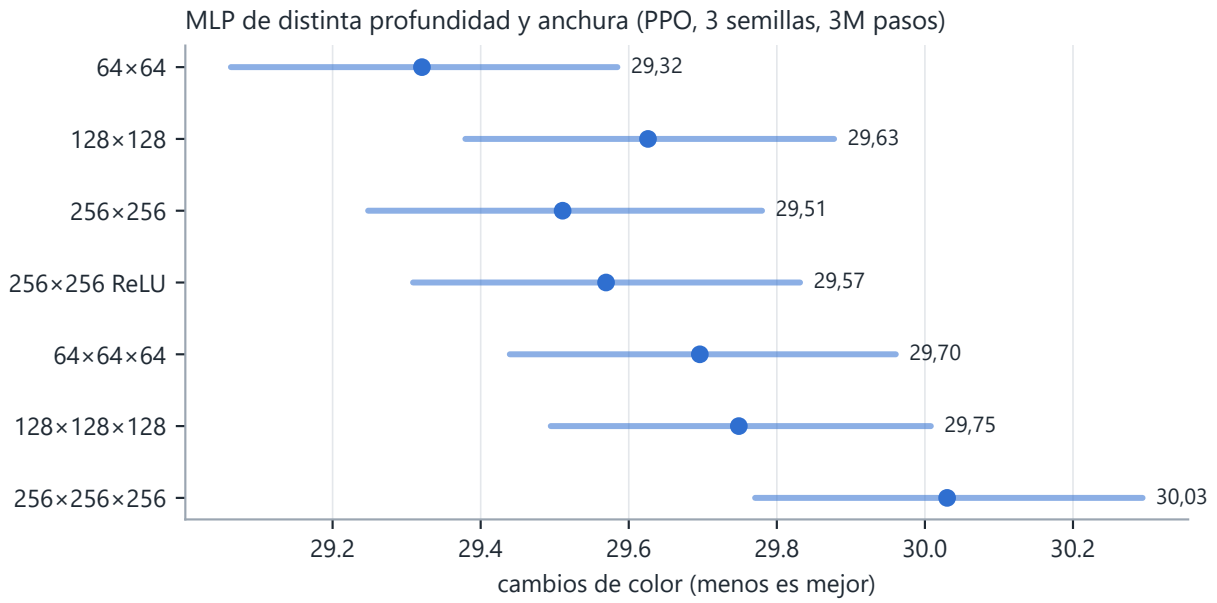


Figura 1. Arquitecturas MLP entrenadas con PPO; media y IC 95 % sobre los escenarios de test ampliados.

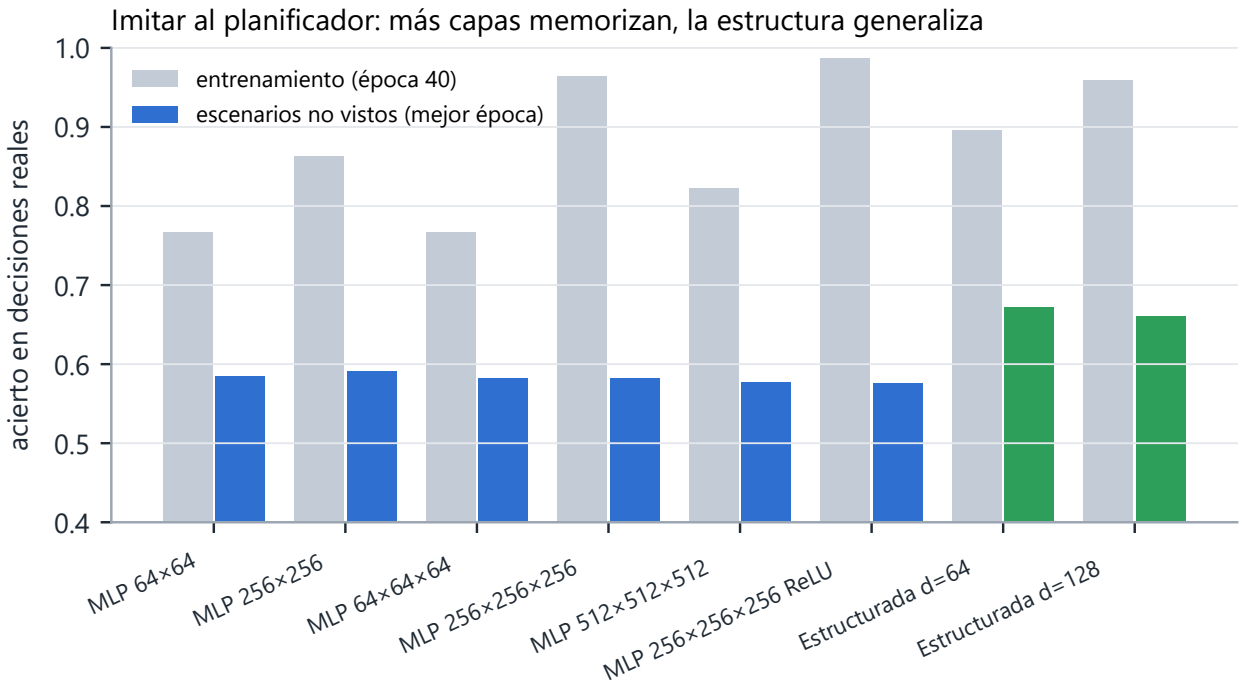


Figura 2. Prueba de imitación: acierto en decisiones no forzadas, entrenamiento frente a escenarios no vistos.

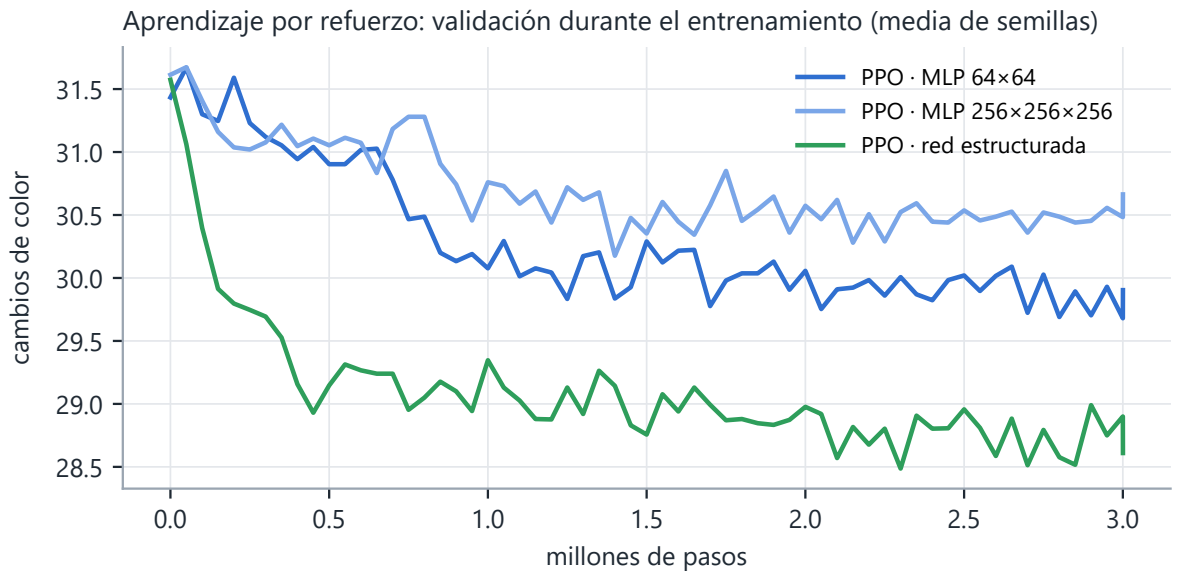


Figura 3. Validación durante el entrenamiento con PPO: la red estructurada aprende en una fracción de los pasos.

5.4 ¿Limita la máscara del paper?

Medimos el techo de cualquier política que obedezca la máscara HAAM restringiendo el planificador a sus acciones: con anticipación 10 alcanza 24,72 cambios y 20,71 con información completa. El agente RL, con la misma información, se queda cerca de 30,09: el cuello de botella era el aprendizaje, no la máscara. Aun así, la máscara sí limita a un alumno bien entrenado: la imitación sin máscara heurística (máscara física) obtiene 25,34 frente a 28,01 con la máscara HAAM.

5.5 Planificar con la información disponible

El planificador con la fórmula de fragmentación, sin ningún entrenamiento, obtiene 22,99 cambios (27,6 %). Ensanchar el haz mejora poco y cuesta mucho: la búsqueda está cerca de saturar y el margen está en cómo se estima el coste futuro.

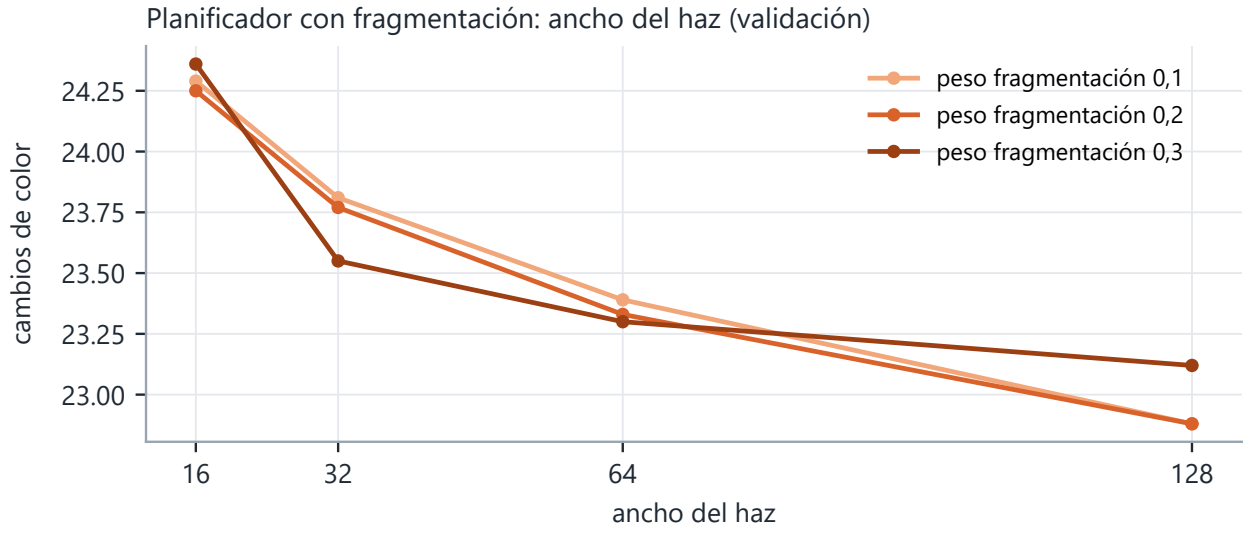


Figura 4. Ancho del haz y peso de la fragmentación del planificador, en los 100 escenarios de validación.

5.6 Varios modelos decidiendo juntos

Probamos tres formas de combinar modelos. La votación del paper no ayuda (sección 5.2). Un comité arbitrado por simulación solo ayuda si el árbitro es bueno: con la fórmula de fragmentación el rollout de la red queda en 29,74 (la red sola: 25,40) y el comité en 23,84, peor que el planificador solo (23,77); con una red de valor como árbitro el rollout de

la red baja a 21,53 y el comité a 17,04 (planificador solo con el mismo juez: 17,10). La combinación que más aporta es el ensemble de jueces: promediar varias redes de valor dentro de la búsqueda.

Tabla 3. Rollouts y comités con dos árbitros distintos (la red sola: 25,40).

Política (validación, 100 escenarios)	Juez de fragmentación	Juez de valor
Rollout de la red	29,74	21,53
Rollout de la heurística	30,99	24,37
Planificador	23,77	17,10
Comité (planificador + 2 rollouts)	23,84	17,04

5.7 Búsqueda guiada por un ensemble de redes de valor

Sustituir la fórmula de fragmentación por una red de valor transforma el planificador. Las 5 redes entrenadas con la misma receta (1200 partidas de el planificador con fragmentación, con semillas distintas) obtienen entre 19,39 y 20,03 cambios en validación cada una. Promediar las 8 redes baja a 17,72 con $\lambda = 0{,}5$ y a 17,10 con $\lambda = 0{,}75$. Tres intentos de mejorar una red individual no funcionaron y se reportan: v3 (3000 partidas de el planificador con fragmentación): 20,29; v2 (1200 partidas de el planificador guiado por v1): 20,78; v4 (3000 partidas de el planificador con fragmentación y la heurística): 21,97. Más datos no garantizan un juez mejor; reentrenar con partidas del propio planificador guiado (iteración de política) lo empeora porque solo ve estados buenos, cuando durante la búsqueda debe juzgar muchos estados malos.

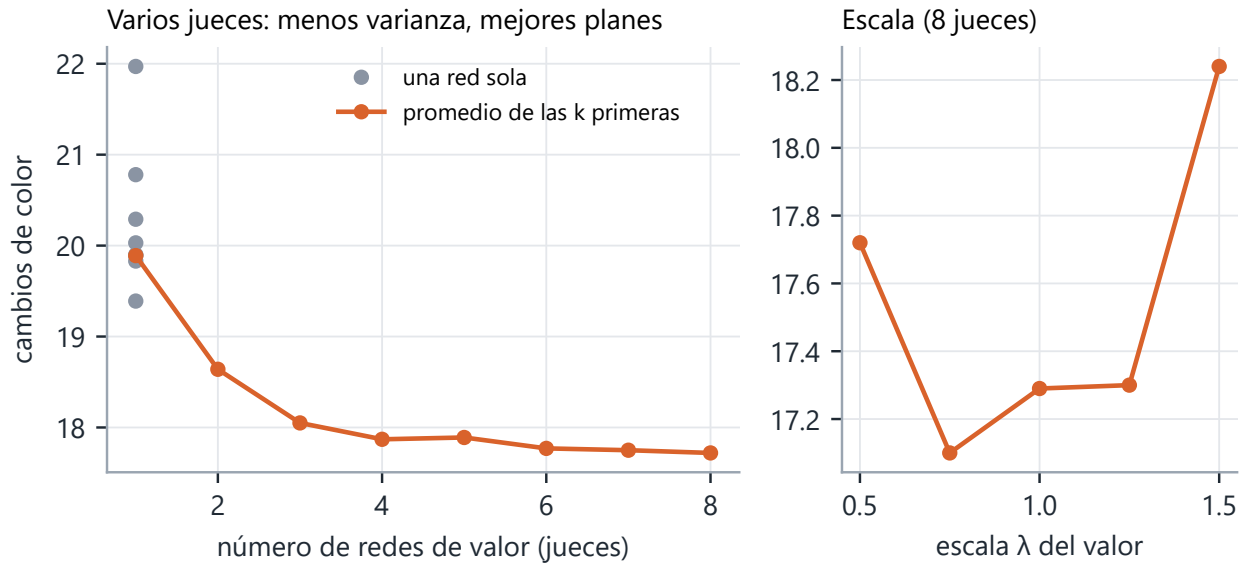


Figura 5. Juez promediado: cambios de color según el número de redes de valor y según la escala λ (validación).

5.8 Destilar el mejor planificador en una red rápida

Para despliegues donde la latencia importe, destilamos el planificador guiado en la red estructurada con DAgger. El alumno obtiene 21,79 cambios (31,3 %) con una sola pasada de red por decisión: mejor que cualquier agente entrenado solo con RL y que el planificador con fragmentación.

5.9 Nivel de llenado de los buffers

Con los buffers llenos el vehículo entrante solo puede ir a la plaza recién liberada. Dejar plazas libres devuelve esa decisión: con 7 plazas libres la heurística baja de 32,33 a 20,40 cambios, y el planificador con fragmentación de 23,27 a 16,53 con 10 plazas libres. Es una decisión de operación de planta, con costes (espacio, ciclo) que el gemelo no modela, y es independiente de la política.

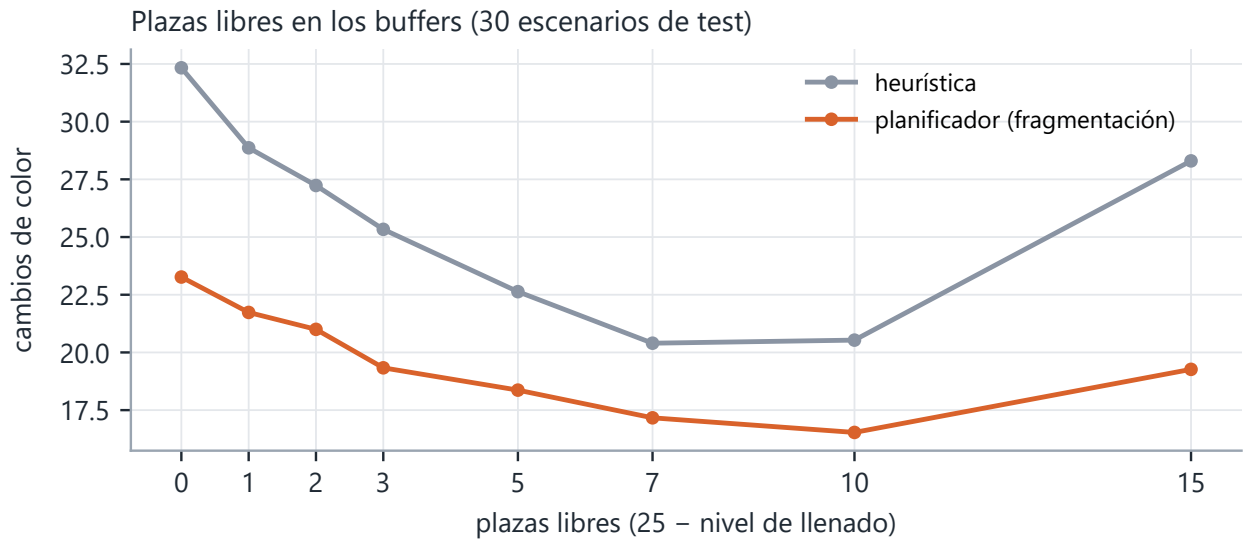


Figura 6. Cambios de color según las plazas que se dejan libres en los buffers.

5.10 ¿Qué pasa si solo se conoce el próximo vehículo?

El agente del paper solo ve el vehículo que espera, el último color pintado y los buffers. Los planificadores de este trabajo pueden ver además la cola: los k vehículos siguientes, en orden («anticipación», propuesta por el paper como trabajo futuro). Todas las cifras anteriores de búsqueda usan k = 10, lo que exige que la planta conozca la secuencia con antelación y que FlexSim la tenga en su cola de espera. Si en la planta solo se conoce el próximo vehículo, hay que usar k = 0. Las redes de valor no ven la cola en ningún caso: solo juzgan el contenido de los buffers, así que sirven en ambos modos.

Tabla 5. La misma política con solo el próximo vehículo conocido y con los 10 siguientes, 500 escenarios de test. Latencia por decisión en un núcleo, sin lotes; la de RL es la de un MLP 64×64. El comité no se evaluó sin cola (su red se entrenó viendo la cola).

Política	Sin cola: cambios	Reducción	G/E/P	ms media (p95)	Cola de 10: cambios	Reducción	G/E/P	ms media (p95)
Heurística LP/CM/UCM	31,73	—	—	0,02 (0,03)	31,73	—	—	0,02 (0,03)
RL · one-hot, MLP 64×64 (5 semillas)	29,62	6,7 %	369/13/118	0,19 (0,23)	30,09	5,2 %	347/9/144	0,19 (0,23)
Planificador con fragmentación	29,97	5,6 %	332/51/117	0,91 (7,5)	22,99	27,6 %	498/2/0	6,28 (13,6)
Planificador guiado por 8 redes de valor	24,34	23,3 %	493/6/1	4,45 (20,2)	16,83	46,9 %	500/0/0	42,86 (116,3)
Comité: planificador + rollouts, juez de valor	—	—	—	—	16,69	47,4 %	500/0/0	54,47 (131,8)

Sin cola, el planificador guiado por valor sigue siendo la mejor política: 24,34 cambios (23,3 %), mejor que la heurística en 493 de 500 escenarios y mejor que cualquier agente RL, incluso los que veían la cola. La fórmula de fragmentación, sin cola, apenas mejora la heurística (29,97): sin llegadas que simular, la calidad del juez lo es todo. El RL tampoco aprovecha la cola (29,62 sin ella, 30,09 con ella). Planificador guiado con cola de 10 frente al mismo sin cola: diferencia media -7,51 cambios (IC 95 % -7,73 a -7,29, $p < 0{,}0001$; mejor en 498, igual en 2 y peor en 0 de 500 escenarios).

Ventajas de cada modo.

- **Sin cola (k = 0).** No necesita conocer la secuencia: sirve en cualquier planta y coincide con la información del paper. Decide en 4,4 ms de media, unas 10 veces más rápido que con cola (p95 20 ms; máximo 127 ms, en el va-

ciado final, cuando ya no llegan vehículos y la búsqueda llega hasta el último). Es la opción por defecto del trigger de FlexSim.

- **Con cola (k = 10).** Casi duplica la mejora (46,9 % frente a 23,3 %), porque puede reservar plazas para los colores que sabe que vienen. Cuesta 43 ms de media (p95 116 ms), todavía muy por debajo de los 5 s de espera de FlexSim, y solo es válido si la secuencia es fiable: si la cola real no coincide con la anunciada, el plan se hace sobre llegadas equivocadas. Admite el comité para una mejora adicional pequeña.

6. Resultados finales

La Figura 7 y la Tabla 4 reúnen todas las familias en los 500 escenarios de test. Comparados directamente, escenario por escenario:

- Comité frente a planificador guiado por valor: diferencia media -0,14 cambios (IC 95 % -0,23 a -0,06, p = 0,0016; mejor en 119, igual en 319 y peor en 62 de 500 escenarios).
- Planificador guiado por valor frente a planificador con fragmentación: diferencia media -6,15 cambios (IC 95 % -6,39 a -5,92, p < 0,0001; mejor en 493, igual en 3 y peor en 4 de 500 escenarios).
- Red destilada frente a planificador con fragmentación: diferencia media -1,19 cambios (IC 95 % -1,46 a -0,93, p < 0,0001; mejor en 296, igual en 60 y peor en 144 de 500 escenarios).
- Imitación física con ajuste PPO frente a la misma sin ajuste: diferencia media -0,26 cambios (IC 95 % -0,44 a -0,09, p = 0,0036; mejor en 258, igual en 39 y peor en 203 de 500 escenarios).
- Planificador guiado por valor frente al oráculo guiado por valor: diferencia media 2,58 cambios (IC 95 % 2,42 a 2,74, p < 0,0001; mejor en 19, igual en 39 y peor en 442 de 500 escenarios).

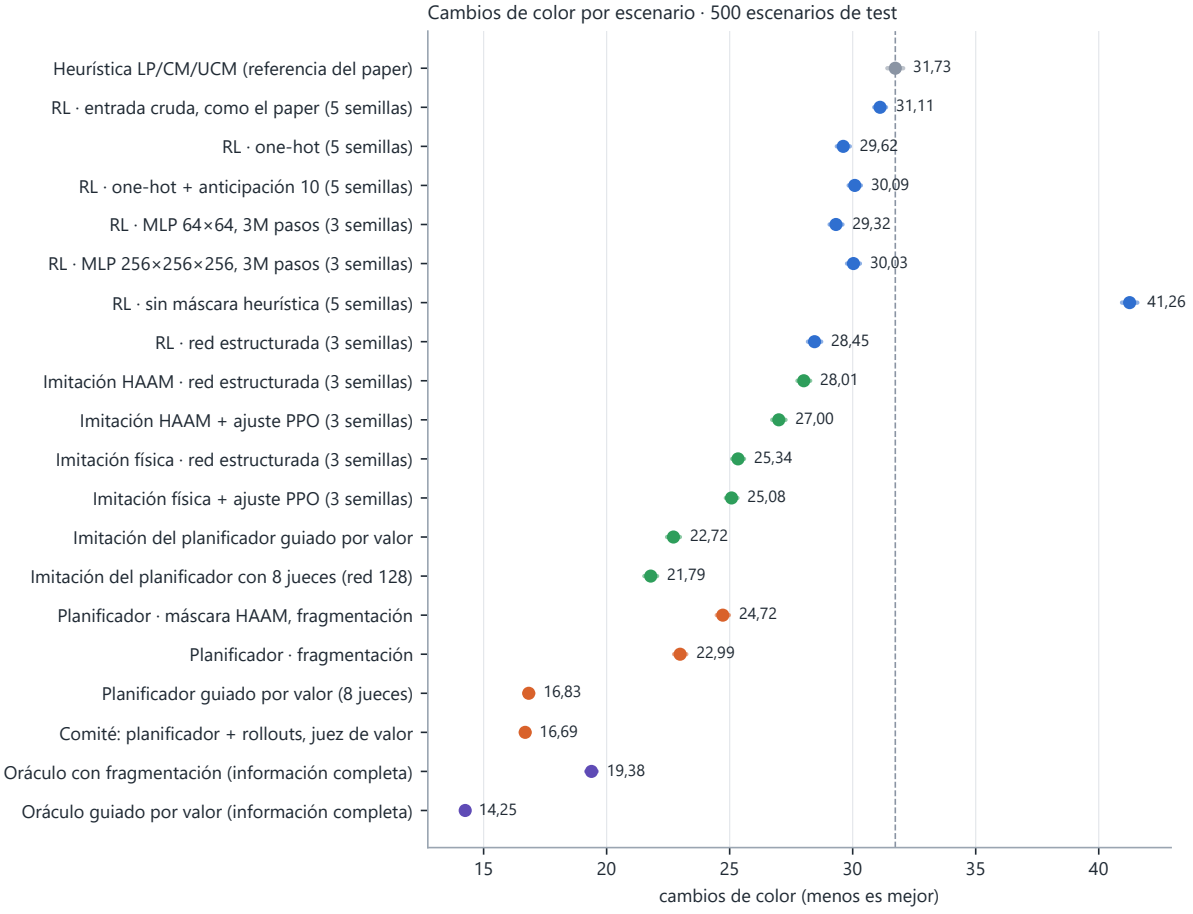


Figura 7. Todas las políticas en 500 escenarios de test: media e IC 95 %. Gris: referencia; azul: RL; verde: imitación; naranja: búsqueda; morado: oráculos con información completa. Línea discontinua: heurística.

Tabla 4. Resultados finales en 500 escenarios de test, pareados contra la heurística; la última columna repite la media en los 30 escenarios del tamaño del paper.

Política	Media	IC 95 %	Reducción	IC reducción	p	G/E/P	Media (30)
Heurística LP/CM/UCM (referencia del paper)	31,73	31,41–32,05	—	—	—	—	32,33
RL · entrada cruda, como el paper (5 semillas)	31,11	30,88–31,35	2,0 %	1,2 a 2,7 %	< 0,0001	291/5/204	31,15
RL · one-hot (5 semillas)	29,62	29,36–29,88	6,7 %	5,9 a 7,4 %	< 0,0001	369/13/118	30,57
RL · one-hot + anticipación 10 (5 semillas)	30,09	29,84–30,33	5,2 %	4,4 a 5,9 %	< 0,0001	347/9/144	30,49
RL · MLP 64×64, 3M pasos (3 semillas)	29,32	29,06–29,58	7,6 %	6,8 a 8,4 %	< 0,0001	381/11/108	29,94
RL · MLP 256×256×256, 3M pasos (3 semillas)	30,03	29,77–30,29	5,4 %	4,6 a 6,2 %	< 0,0001	346/15/139	30,10
RL · sin máscara heurística (5 semillas)	41,26	40,95–41,57	-30,0 %	-31,1 a -28,9 %	< 0,0001	0/1/499	41,19
RL · red estructurada (3 semillas)	28,45	28,18–28,72	10,3 %	9,5 a 11,1 %	< 0,0001	416/12/72	29,01
Imitación HAAM · red estructurada (3 semillas)	28,01	27,76–28,27	11,7 %	11,0 a 12,5 %	< 0,0001	437/16/47	28,40
Imitación HAAM + ajuste PPO (3 semillas)	27,00	26,73–27,27	14,9 %	14,2 a 15,7 %	< 0,0001	463/10/27	27,73
Imitación física · red estructurada (3 semillas)	25,34	25,10–25,58	20,1 %	19,4 a 20,9 %	< 0,0001	489/5/6	25,59
Imitación física + ajuste PPO (3 semillas)	25,08	24,85–25,32	21,0 %	20,2 a 21,7 %	< 0,0001	492/1/7	25,23
Imitación del planificador guiado por valor	22,72	22,46–22,98	28,4 %	27,6 a 29,2 %	< 0,0001	496/3/1	22,63
Imitación del planificador con 8 jueces (red 128)	21,79	21,53–22,05	31,3 %	30,5 a 32,1 %	< 0,0001	500/0/0	21,83
Planificador · máscara HAAM, fragmentación	24,72	24,48–24,97	22,1 %	21,3 a 22,9 %	< 0,0001	493/4/3	24,63
Planificador · fragmentación	22,99	22,74–23,24	27,6 %	26,8 a 28,4 %	< 0,0001	498/2/0	23,27
Planificador guiado por valor (8 jueces)	16,83	16,65–17,02	46,9 %	46,4 a 47,5 %	< 0,0001	500/0/0	16,63
Comité: planificador + rollouts, juez de valor	16,69	16,52–16,87	47,4 %	46,8 a 48,0 %	< 0,0001	500/0/0	16,77
Oráculo con fragmentación (información completa)	19,38	19,17–19,60	38,9 %	38,2 a 39,6 %	< 0,0001	500/0/0	19,33
Oráculo guiado por valor (información completa)	14,25	14,11–14,40	55,1 %	54,6 a 55,6 %	< 0,0001	500/0/0	14,47

7. Coste por decisión

Tabla 6. Latencia por decisión en un núcleo de CPU, sin procesamiento por lotes; h es la anticipación (h=0: solo el próximo vehículo).

Política	Media (ms)	p95 (ms)	Máximo (ms)
heuristic	0,02	0,03	0,39
planner(h=10,w=32)	6,28	13,57	18,15
planner(h=10,w=32,mask=haam)	2,28	5,89	12,17
planner(h=20,w=32)	12,30	21,81	27,93
planner(h=20,w=32,mask=haam)	6,32	11,59	22,88
artifacts/runs/bufferpair128_physical_judgesdagger/seed_0/best_model.zip	0,42	0,50	0,58
artifacts/runs/arch_64x64/seed_0/best_model.zip	0,19	0,23	0,30
value_planner(h=10,w=32,scale=0.75,judges=8)[v1+v5+v6+v7+v8+v3+v2+v4]	42,86	116,29	206,98
committee[v1+v5+v6+v7+v8+v3+v2+v4]	54,47	131,77	230,54
planner(h=0,w=32)	0,91	7,52	11,54
planner(h=0,w=32,mask=haam)	0,26	1,81	5,80
value_planner(h=0,w=32,scale=0.75,judges=8)[v1+v2+v3+v4+v5+v6+v7+v8]	4,45	20,23	126,65

Todas las políticas caben en el tiempo de decisión de FlexSim (5 s); la red destilada es dos órdenes de magnitud más rápida que el planificador guiado, y el planificador guiado sin cola, uno.

8. Discusión

Por qué RL rinde poco aquí. Las consecuencias de una decisión se ven decenas de pasos después y el 55 % de las decisiones con máscara HAAM son forzadas; el agente recibe poca señal útil. Más capas no lo resuelven: los MLP grandes memorizan sin generalizar. Lo que sí ayuda es incorporar la estructura del problema (equivarianza entre buffers) y aprender de un experto (imitación) en lugar de solo por ensayo y error.

Por qué gana la búsqueda guiada por valor. Separa lo que se sabe (las llegadas conocidas, que se simulan exactamente) de lo que no (el futuro, que estima una red). La calidad del juez es decisiva: con una buena estimación, incluso combinar modelos por simulación funciona; con una mala, empeora. Promediar jueces es la forma más barata y robusta de mejorarlo.

Recomendación. Desplegar el planificador guiado por el ensemble de redes de valor. Si la planta conoce la secuencia de los 10 vehículos siguientes, con anticipación 10 (16,83 cambios, 46,9 % menos que la heurística), que decide en 43 ms de media; el comité con rollouts añade una mejora pequeña pero significativa si se dispone de algo más de tiempo por decisión. Si solo se conoce el próximo vehículo, desplegarlo sin anticipación (24,34 cambios, 23,3 % menos, 4,4 ms de media). Si la latencia fuera crítica, la versión destilada del planificador (21,79 cambios) decide en menos de un milisegundo. Revisar además el nivel de llenado de los buffers con producción.

9. Limitaciones y trabajo futuro

- Los resultados son del gemelo en Python. El verificador de trazas permite confirmar en FlexSim, escenario por escenario, que ambos simulan lo mismo; ese paso está pendiente.
- Un solo tipo de escenario (100 vehículos, mezcla del paper). Faltan escenarios más largos, otras mezclas y restricciones reales (limpiezas, costes por par de colores).
- El oráculo es una cota superior del óptimo, no el óptimo; un solver exacto en instancias pequeñas daría la brecha real.
- Siguientes pasos: entrenar las redes de valor con estados generados por la propia búsqueda, búsqueda Monte Carlo (MCTS) con el mismo juez y decidir el nivel de llenado de forma dinámica.

10. Reproducibilidad

```
pip install -e ".[dev]" && pytest
python scripts/run_experiments.py --configs configs/*.json --seeds 0 1 2 3 4
python scripts/distill.py --config configs/onehot_lookahead_physical.json --teacher-value ...
python scripts/train_value.py --out artifacts/value/v1.pt
python scripts/research/value_judges.py --values artifacts/value/v*.pt
python scripts/research/final_evaluation.py --value <redes> --committee-network <red>
python scripts/evaluate.py --seed-set extended --planner 0 --value <redes> --value-horizon 0 --out
artifacts/eval/horizon0_extended.json
python scripts/benchmark.py --planner-horizons 0 --value <redes> --value-horizons 0 10 --out
artifacts/eval/latency_horizon0.json
python scripts/build_publication.py
```

Referencias

1. Choi, K., Rho, C., Kim, T. y Choi, D. (2024). Heuristic Algorithm-based Action Masking Reinforcement Learning (HAAM-RL) with Ensemble Inference Method. arXiv:2403.14110.
2. Krystek, J. y Bysko, S. (2019). Virtual commissioning as the main core of Industry 4.0 – case study in the automotive paint shop.
3. Ng, A. Y., Harada, D. y Russell, S. (1999). Policy invariance under reward transformations. ICML.
4. Schulman, J. et al. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.
5. Ross, S., Gordon, G. y Bagnell, D. (2011). A reduction of imitation learning and structured prediction to no-regret online learning (DAgger). AISTATS.
6. Bertsekas, D. (2019). Reinforcement Learning and Optimal Control (algoritmos de rollout). Athena Scientific.
7. Silver, D. et al. (2017). Mastering the game of Go without human knowledge. Nature 550.
8. Zaheer, M. et al. (2017). Deep Sets. NeurIPS.
9. Huang, S. y Ontañón, S. (2022). A Closer Look at Invalid Action Masking in Policy Gradient Algorithms. FLAIRS.

Apéndice A. Redes de valor

Tabla A1. Redes de valor entrenadas; el planificador final promedia todas.

Red	Partidas de entrenamiento	R^2 (partidas no vistas)	Escenarios
v1	Partidas de el planificador con fragmentación	0,662	1.200
v2	Partidas de el planificador guiado por v1	0,564	1.200
v3	Partidas de el planificador con fragmentación	0,676	3.000
v4	Partidas de el planificador con fragmentación y la heurística	0,508	3.000
v5	Partidas de el planificador con fragmentación	0,675	1.200
v6	Partidas de el planificador con fragmentación	0,697	1.200
v7	Partidas de el planificador con fragmentación	0,688	1.200
v8	Partidas de el planificador con fragmentación	0,646	1.200